

第 6 讲

Data Locality and Tiled Matrix Multiply

目录

1	课程事项与核心内容	1
1.1	学习目标与课程事项	1
1.2	Delta 命令行支持工具	1
2	数据局部性与分块矩阵乘法	2
2.1	从内积计算到输入复用	2
2.2	局部性与共享内存	4
2.3	沿内积维度推进分块	5
3	线程块同步与正确执行顺序	8
3.1	共享数据带来的两类依赖	8
3.2	屏障与整体同步执行	9
3.3	整除情形的完整核函数	10
4	边界条件与一般尺寸矩阵	11
4.1	从矩阵形状推导三类有效性条件	11
4.2	零填充维持统一的计算循环	13
4.3	支持一般尺寸的完整实现	16
5	分块收益与资源约束	18
5.1	用数据流量解释计算收益	18
5.2	带宽瓶颈如何转向计算瓶颈	19
5.3	分块尺寸与驻留资源	20
6	共享内存接口与分块思想的推广	22
6.1	动态共享内存	22
6.2	Hopper 的相关功能	23
6.3	CPU 缓存分块	23
7	关键结论与公式汇总	25
7.1	机制之间的联系	25
7.2	关键公式	25

1 课程事项与核心内容

1.1 学习目标与课程事项

学习目标为：为 Lab 3 的分块矩阵乘法做准备，处理分块算法的边界条件，以及评估分块带来的收益。技术内容围绕同一个计算问题展开：将矩阵乘法中的输入复用转化为较少的全局内存访问，同时保证线程协作的正确性。

1.1.1 作业与阅读安排

以下课程事项对应 2026 年 9 月 10 日。Lab 1 在当周周五截止，要求先完成 Lab 0，且 Lab 1 不计入最终成绩。阅读材料为教材第 5 章；待办事项包括获取 Delta 账号、设置 GitHub，以及提交 Labs 0 和 1。

实验和项目的通常截止时间为周五 20:00，默认提供三天宽限期：周一 20:00 前提交仍可获得完整分数，无需提前通知。若需要超过三天的宽限期，必须在周五 20:00 前，通过邮件向负责相应作业的助教提出申请；是否批准取决于具体特殊情况，超过这一申请时间的请求不予考虑。最低分的一次实验会从总评中剔除；在上述延期政策之外，实验和项目不再接受逾期提交。

1.1.2 概念关系

全局内存带宽限制了数据供给速度；共享内存使线程块可以显式保存并复用一小块输入。矩阵乘法沿内积维度逐块累加，形成“协作加载—同步—局部计算—同步”的循环。边界处理维持这一循环的统一结构，性能分析则比较数据复用与线程块资源消耗。

1.2 Delta 命令行支持工具

1.2.1 启动与用途

命令行支持代理可协助排查 Delta 使用问题，例如实验无法运行、SLURM 作业状态查询，也可辅助实验代码编写。使用前查看 lab1 目录下的 hpcGPT.md。启动命令为：

```
module load hpc-gpt/1.18.23-1
opencode
```

该模块版本对应 2026 年 9 月 10 日的支持工具入口。

内存分配示例涉及三个数组：两个输入和一个输出都需要对应的设备端空间。分配量由元素个数乘以 `sizeof(float)` 得到，单位是字节；CUDA 分配函数需要设备指针的地址，以便把分配结果写回该指针。

1.2.2 内置命令

输入 `/` 可以调出内置命令列表。下表按用途归类保留命令及其含义。

用途	命令与含义
会话与目录	<code>/new</code> 新会话； <code>/sessions</code> 切换会话； <code>/move</code> 更换项目目录； <code>/exit</code> 退出。
代理与连接	<code>/agents</code> 切换代理； <code>/models</code> 切换模型； <code>/connect</code> 连接提供方； <code>/mcps</code> 切换 MCP。
作业与状态	<code>/jobs</code> 显示或隐藏侧栏 SLURM 作业； <code>/jobs-refresh</code> 立即刷新作业； <code>/status</code> 查看状态。
编辑与检查	<code>/editor</code> 打开编辑器； <code>/diff</code> 查看差异； <code>/review</code> 检查变更，支持 <code>commit</code> 、 <code>branch</code> 、 <code>pr</code> ，默认检查未提交变更。
配置与帮助	<code>/init</code> 引导设置 AGENTS.md； <code>/skills</code> 查看技能； <code>/themes</code> 主题； <code>/help</code> 帮助。
诊断与反馈	<code>/debug</code> 调试信息； <code>/feedback</code> 附带当前对话提交反馈； <code>/report</code> 向 Delta 支持团队报告问题。

2 数据局部性与分块矩阵乘法

2.1 从内积计算到输入复用

2.1.1 矩阵、线程与下标约定

先考虑两个 $N \times N$ 矩阵相乘， $C = AB$ 。采用与数组一致的零起始下标，有

$$C_{r,c} = \sum_{j=0}^{N-1} A_{r,j} B_{j,c}, \quad 0 \leq r, c < N. \quad (1)$$

每个输出元素是 A 的一行与 B 的一列的内积。基本线程映射是：一个线程负责一个 $C_{r,c}$ ，在线程私有的累加变量中保存部分和，内积完成后写回一次。

令 T 表示方形分块的边长，对应代码中的 `TILE_DIM` 或 `TILE_WIDTH`。一个线程块含 $T \times T$ 个线程，负责一个同样大小的输出分块。线程块坐标为 (b_x, b_y) ，块内线程坐标为 (t_x, t_y) ；

横坐标对应列，纵坐标对应行：

$$r = b_y T + t_y, \quad c = b_x T + t_x, \quad 0 \leq t_x, t_y < T. \quad (2)$$

后续使用 q 表示沿内积方向的分块编号， k 表示某一分块内部的内积下标，因此完整内积下标为 $j = qT + k$ 。这三个层次分别回答“哪个输出块”“哪个输出元素”和“当前使用哪一段输入”。

2.1.2 同一个输入服务于多个输出

固定 r, j ，元素 $A_{r,j}$ 参与 C 的第 r 行全部 N 个输出；固定 j, c ，元素 $B_{j,c}$ 参与 C 的第 c 列全部 N 个输出。因此，输入矩阵中每个元素都具有重复使用的机会。

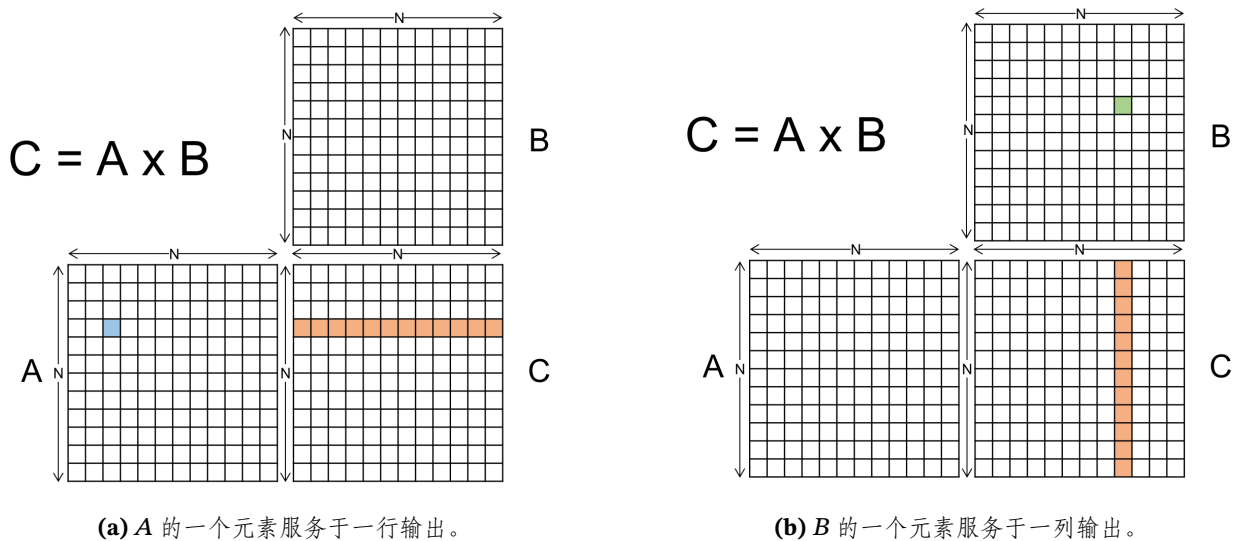


图 1 矩阵乘法的输入复用关系。

当一个线程块负责 $T \times T$ 个输出时，同一行的 T 个线程会使用相同的 A 元素，同一列的 T 个线程会使用相同的 B 元素。将这些共同需要的输入保存在块内共享的存储区域，就能让一次全局加载支撑多次计算。

2.2 局部性与共享内存

2.2.1 带宽为什么会限制运算速度

全局内存 (*global memory*) 由 DRAM 实现。线程读取的数据有时已经位于 L1 缓存中；也可能在另一个线程再次使用之前被逐出。**共享内存** (*shared memory*) 为线程块提供一个显式管理的、较小而较快的数据区域，可以直接组织所需的复用。

数据局部性 (*data locality*) 在这里体现为：把一组即将反复访问的数据集中起来，在它们仍然可用时完成尽可能多的相关计算。按一次乘法和一次加法合计两个浮点运算计数，每个内积步读取两个 `float` 输入，需要 8 字节，得到

$$\frac{8 \text{ B}}{2 \text{ FLOP}} = 4 \text{ B/FLOP}. \quad (3)$$

暂时忽略输出写回和缓存复用。对于计算峰值 1000 GFLOP/s、内存带宽 150 GB/s 的示例 GPU，内存只能支撑

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s}. \quad (4)$$

计算单元即使具有更高峰值，也需要足够快的数据供给才能发挥作用。这组参数说明了带宽对数据供给的限制。

2.2.2 分块使数据复用成为显式协作

分块 (*tiling*) 将数据划分为能够放入共享内存的子集。线程块协作把当前子集从全局内存搬入共享内存，随后反复使用它完成计算，再处理下一组输入。在基本分块矩阵乘法中，每个线程每轮加载一个 A 元素和一个 B 元素，整个线程块共同填满两个 $T \times T$ 的共享数组。

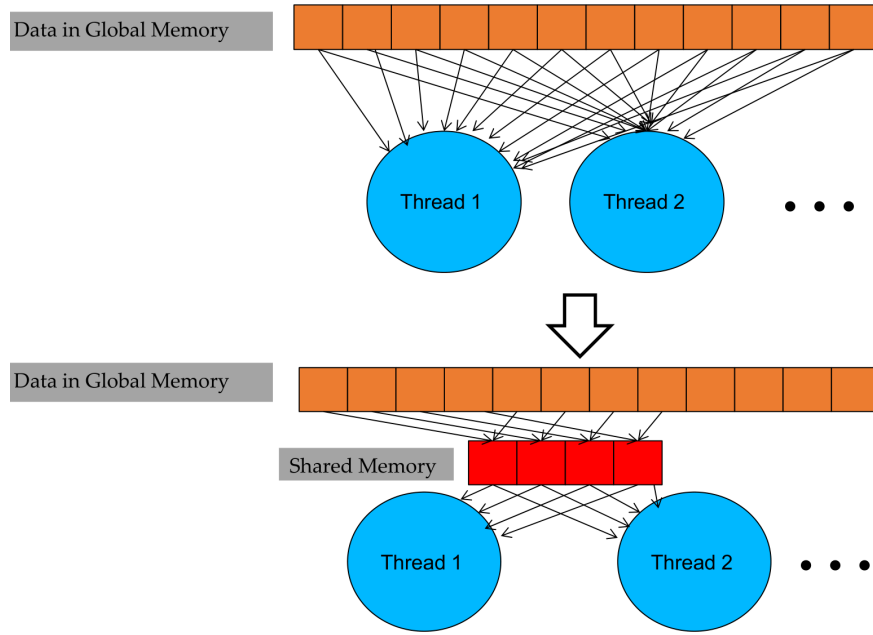


图 2 多线程直接重复读取全局数据，以及经共享内存组织复用的两种访问方式。

数据的存放位置

A 、 B 的完整输入和 C 的完整输出位于全局内存；当前两个输入分块位于共享内存；每个线程的部分和 sum 位于寄存器中。一个输出块的结果由该块全部线程的私有部分和共同构成。计算结束时，各线程把自己的最终结果直接写回 C 。

2.3 沿内积维度推进分块

2.3.1 固定输出块，依次更换输入块

先假定 N 是 T 的整数倍。输出块的 (b_x, b_y) 在整个核函数执行中保持不变，而输入分块随 $q = 0, 1, \dots, N/T - 1$ 推进。第 q 轮使用 A 的列区间 $[qT, (q+1)T)$ 和 B 的行区间 $[qT, (q+1)T)$ 。

把式 (1) 的求和下标写成 $j = qT + k$ ，得到与分块循环直接对应的等式：

$$C_{r,c} = \sum_{q=0}^{N/T-1} \left(\sum_{k=0}^{T-1} A_{r,qT+k} B_{qT+k,c} \right). \quad (5)$$

内层求和产生一个部分内积，外层求和累加所有输入分块的贡献。此重排用于说明分块算法与原矩阵乘法的对应关系。

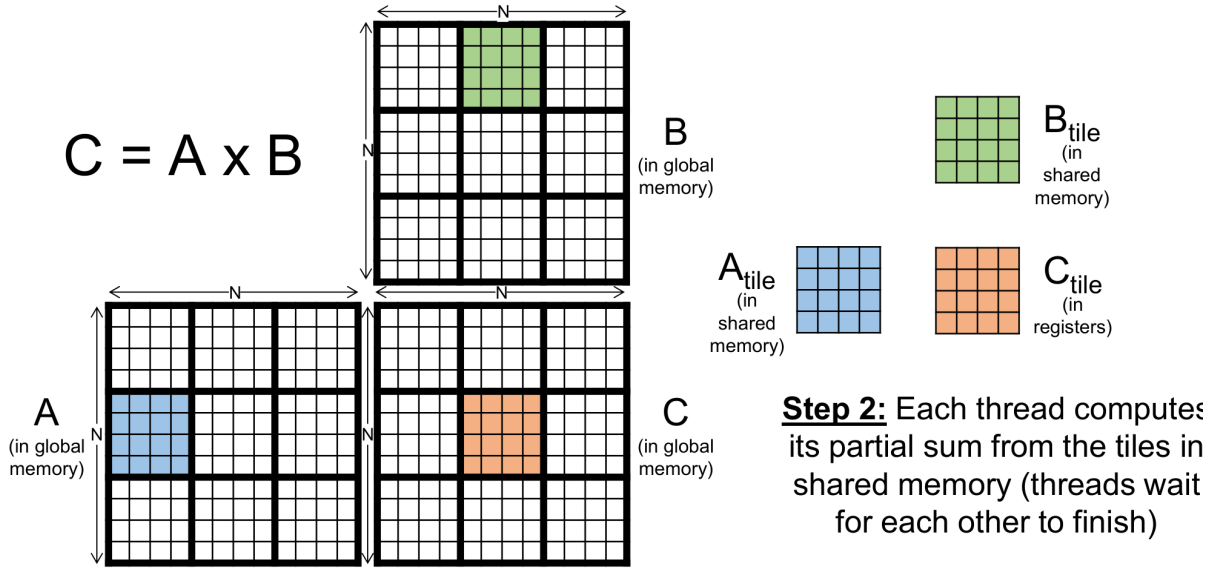


图 3 一个输出块对应一对当前输入块；输入块位于共享内存，输出部分和位于各线程寄存器。

2.3.2 协作加载的二维索引

线程 (t_x, t_y) 把两个输入元素分别写入共享数组的同一个局部位置 $[t_y][t_x]$:

$$A_s[t_y][t_x] \leftarrow A[r][qT + t_x], \quad (6)$$

$$B_s[t_y][t_x] \leftarrow B[qT + t_y][c]. \quad (7)$$

对于 A ，全局行号固定在当前输出行，沿列方向推进分块；对于 B ，全局列号固定在当前输出列，沿行方向推进分块。因此， A 的内积方向由 t_x 定位， B 的内积方向由 t_y 定位。

第 0 轮分别读取 $A[r][t_x]$ 、 $B[t_y][c]$ ；第 1 轮分别读取 $A[r][T + t_x]$ 、 $B[T + t_y][c]$ 。块内所有线程联合覆盖整对输入分块。加载阶段还应使同一 warp 中的访问便于合并访问 (coalesced access)。

2.3.3 行主序的一维地址

行主序 (row-major order) 将每一行连续存放。对于列数为 N 的数组，二维位置 (i, j) 的一维偏移为 $iN + j$ 。式 (7) 因而变为：

```
A_s[ty][tx] = A[row * N + q * TILE_DIM + tx];
B_s[ty][tx] = B[(q * TILE_DIM + ty) * N + col];
```

A 中的 $qT + t_x$ 已经是列偏移，无需再乘以行宽； B 中的 $qT + t_y$ 是行号，必须先乘以行宽。共享数组在声明时具有二维形状，可以继续使用二维下标。

Loading an Input Tile q

- However, recall that A and B are commonly stored as flat row-major arrays and can only use 1D indexing

```
A[Row][q*TILE_WIDTH+tx]
A[Row*Width + q*TILE_WIDTH + tx]

B[q*TILE_WIDTH+ty][Col]
B[(q*TILE_WIDTH+ty) * Width + Col]
```

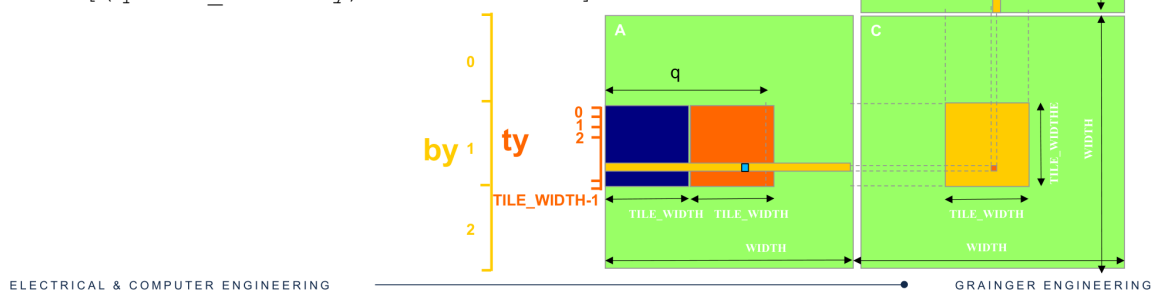


图 4 输出块坐标固定， A 的输入块向右移动， B 的输入块向下移动；两者由同一分块编号 q 控制。

2.3.4 计算时读取共享行与共享列

分块加载完成后，线程 (t_x, t_y) 按自己的输出位置读取共享数组的一行与一列：

```
for (int k = 0; k < TILE_DIM; ++k) {
    sum += A_s[ty][k] * B_s[k][tx];
}
```

将加载定义代入可验证，每一步使用的恰好是

$$A_s[t_y][k]B_s[k][t_x] = A_{r,qT+k}B_{qT+k,c}$$

加载时，线程负责 $[t_y][t_x]$ 这个存储位置；计算时，它需要由多个线程加载的共享行和共享列。正是这一区分使协作加载产生复用，也使线程间同步成为必要条件。

例：一个线程在加载与计算中的不同职责

取 $N = 4$ 、 $T = 2$ ，考虑线程块 $(b_x, b_y) = (0, 1)$ 中的线程 $(t_x, t_y) = (1, 0)$ 。由式 (2)，它负责 $C_{2,1}$ 。

在 $q = 0$ 时，该线程加载 $A_{2,1}$ 与 $B_{0,1}$ ，但其部分内积为

$$A_{2,0}B_{0,1} + A_{2,1}B_{1,1}.$$

其中 $A_{2,0}$ 和 $B_{1,1}$ 由其他线程加载。在 $q = 1$ 时，该线程加载 $A_{2,3}$ 与 $B_{2,1}$ ，并累加

$$A_{2,2}B_{2,1} + A_{2,3}B_{3,1}.$$

两轮得到完整的 $C_{2,1}$ 。该例用于逐项解释加载索引和计算索引，并沿用每线程一个输出的基本算法。

3 线程块同步与正确执行顺序

3.1 共享数据带来的两类依赖

3.1.1 使用之前，必须完成当前分块加载

每个线程只加载当前输入分块的一小部分，却会在内积循环中读取其他线程负责的元素。例如，线程 (t_x, t_y) 在第 k 步读取的 $A_s[t_y][k]$ ，由线程 (k, t_y) 加载； $B_s[k][t_x]$ ，由线程 (t_x, k) 加载。开始计算之前，必须保证这些加载已经完成。

3.1.2 覆盖之前，必须完成上一分块使用

两轮分块复用相同的共享数组 A_s 、 B_s 。较快的线程如果提前进入下一轮，就可能把第 $q + 1$ 轮数据写到仍被其他线程读取的位置。这会让某些乘积混用两轮输入。进入下一轮加载之前，需要确认当前轮的读取和计算已经结束。

这两类依赖分别保护“生产完成后再消费”和“消费完成后再覆盖”。只等待加载完成，不能保护共享数组的再次使用。

3.2 屏障与整体同步执行

3.2.1 屏障的含义

屏障 (barrier) 是线程集合共同到达的同步点。先到达的线程等待其余线程；参与线程全部到达后，才能继续执行屏障之后的代码。

整体同步执行 (bulk synchronous execution) 将并行计算划分为若干阶段：每个阶段内部，各线程并行完成自己的工作；阶段之间由屏障建立清晰的先后关系。线程在一个阶段内部的执行进度可以不同。

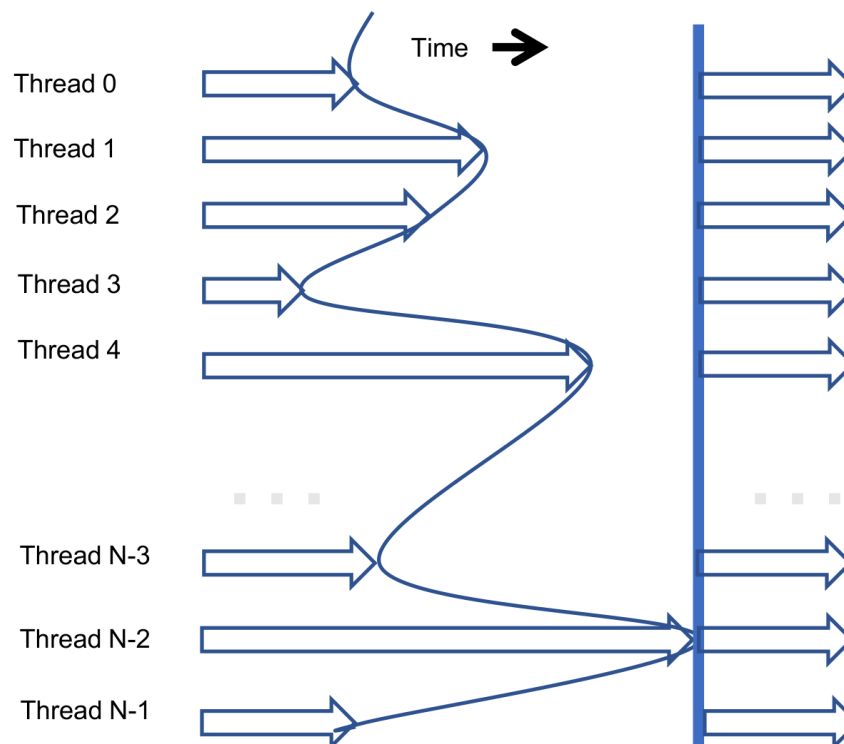


图 5 线程以不同进度到达屏障，在共同的同步点之后继续执行。

3.2.2 CUDA 的块内同步

`__syncthreads()` 是 CUDA 的设备端内置同步操作，用于同一线程块中的线程。线程块内的所有线程需要到达相应同步点，才能继续执行。

同步调用可以位于条件分支内，前提是该条件对整个参与线程块一致。如果只有一部分线程执行同步，执行可能挂起或产生错误行为。边界线程处理尤其需要保持这一点：线程的输入加载是否有效可以不同，但参与分块循环和同步的结构应保持一致。

每一轮的执行顺序

协作加载 $A_s, B_s \rightarrow \text{__syncthreads}()$

当前分块的部分内积 $\rightarrow \text{__syncthreads}()$

然后进入下一轮加载。

第一个屏障保证“数据已准备好”，第二个屏障保证“旧数据已使用完”。屏障作用于当前线程块；不同输出块具有各自的输入缓冲区和输出区域。

3.3 整除情形的完整核函数

下面将加载、两次同步、部分内积和写回组合起来。假设 $N > 0$ 且 T 整除 N ，所有矩阵均按行主序存储；线程块大小固定为 $T \times T$ ，每线程负责一个输出。代码将 `TILE_DIM` 设为 16，这一取值仅用于具体化示例。

代码 1 带两次块内同步的方阵分块核函数。

```

1  #include <cuda_runtime.h>
2  #define TILE_DIM 16
3
4  __global__ void matmul_tiled_square(
5      const float* A, const float* B, float* C, int N)
6  {
7      __shared__ float A_s[TILE_DIM][TILE_DIM];
8      __shared__ float B_s[TILE_DIM][TILE_DIM];
9      const int tx = threadIdx.x;
10     const int ty = threadIdx.y;
11     const int row = blockIdx.y * TILE_DIM + ty;
12     const int col = blockIdx.x * TILE_DIM + tx;
13     float sum = 0.0f;
14
15     for (int q = 0; q < N / TILE_DIM; ++q) {
16         A_s[ty][tx] = A[row * N + q * TILE_DIM + tx];
17         B_s[ty][tx] = B[(q * TILE_DIM + ty) * N + col];
18         __syncthreads(); // Wait for all tile producers.
19
20         for (int k = 0; k < TILE_DIM; ++k) {
21             sum += A_s[ty][k] * B_s[k][tx];
22         }
23         __syncthreads(); // Wait for all tile consumers.
24     }
25     C[row * N + col] = sum;
26 }
```

对应的主机端启动代码为：

```
dim3 block(TILE_DIM, TILE_DIM);
dim3 grid(N / TILE_DIM, N / TILE_DIM);
matmul_tiled_square<<<grid, block>>>(d_A, d_B, d_C, N);
```

其中 d_A 、 d_B 、 d_C 是已经分配的设备端矩阵指针。启动片段只说明线程配置，沿用已完成输入准备和设备内存分配的前提。

3.3.1 正确性的阶段不变量

以下不变量用于形式化说明代码 1 的执行过程。第 q 轮开始时，线程 (t_x, t_y) 的累加值为

$$\text{sum} = \sum_{j=0}^{qT-1} A_{r,j} B_{j,c}.$$

第一次同步之后，当前所需的共享行与共享列均已加载。内积循环增加

$$\sum_{k=0}^{T-1} A_{r,qT+k} B_{qT+k,c},$$

所以本轮结束时，累加范围恰好扩展为 $0 \leq j < (q+1)T$ 。第二次同步保证当前输入不会被过早覆盖。全部 N/T 轮结束后，累加范围覆盖 $0 \leq j < N$ ，因而 sum 等于 $C_{r,c}$ 。

4 边界条件与一般尺寸矩阵

4.1 从矩阵形状推导三类有效性条件

4.1.1 一般矩阵尺寸与行宽

采用一般矩阵形状

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}. \quad (8)$$

其中 M 是输出行数， N 是输出列数， K 是内积长度。方阵情形对应 $M = K = N$ 。线程映射仍为 $r = b_y T + t_y$ 、 $c = b_x T + t_x$ 。

A 的行宽为 K , B 和 C 的行宽为 N 。因此, 一维数组偏移分别为

$$A_{r,j} \longleftrightarrow rK + j, \quad B_{j,c} \longleftrightarrow jN + c, \quad C_{r,c} \longleftrightarrow rN + c.$$

各矩阵的行宽决定地址偏移; 加载条件和输出条件需要分别使用相应矩阵的维度。

4.1.2 加载有效性与输出有效性分别判断

同一个线程在一轮中有三个独立的任务: 加载一个 A 元素、加载一个 B 元素, 以及更新自己的输出部分和。前两项是否对应有效输入, 需要根据各自的输入坐标判断。

操作	目标位置	有效性条件
加载 A	$A_{r,qT+t_x}$	$r < M$ 且 $qT + t_x < K$ 。
加载 B	$B_{qT+t_y,c}$	$qT + t_y < K$ 且 $c < N$ 。
写回 C	$C_{r,c}$	$r < M$ 且 $c < N$ 。

所有下标由非负线程坐标生成, 因此这里仅列上界条件。

一个输出列越界的线程, 仍可能负责加载同块其他线程需要的 A 元素; 一个输出行越界的线程, 仍可能负责加载有效的 B 元素。输出有效性只能决定是否写回当前输出, 不能直接代替输入加载条件。

4.1.3 三种方向的分块数量

输出网格覆盖 $M \times N$, 所以纵横方向的线程块数分别为

$$n_x = \left\lceil \frac{N}{T} \right\rceil, \quad n_y = \left\lceil \frac{M}{T} \right\rceil. \quad (9)$$

每个输出都需要遍历长度为 K 的内积方向, 因此每个线程块执行

$$Q = \left\lceil \frac{K}{T} \right\rceil \quad (10)$$

轮输入加载。网格尺寸由输出矩阵决定, 循环轮数由内积长度决定。

对于正整数 D , 整数向上取整可以写成

$$\left\lceil \frac{D}{T} \right\rceil = \left\lfloor \frac{D-1}{T} \right\rfloor + 1. \quad (11)$$

这对应 $(D - 1) / \text{TILE_DIM} + 1$ 。若 D 为 T 的倍数，减一使商减少一，再加一恢复；若存在余数，则在完整分块数上加一，覆盖最后的不完整分块。公式用于 $D > 0$ 的情形。

4.2 零填充维持统一的计算循环

4.2.1 缺失输入在加载时写零

零填充 (zero padding) 在加载阶段处理边界：目标输入有效时正常加载，越界时向该线程负责的共享数组位置写入 0。随后，所有线程都可以执行相同的 T 步内积循环。超出内积长度的项因乘以零而不产生贡献。

每一轮都必须写入该共享位置，无论写入的是输入还是零。共享数组会在多轮之间复用；若越界分支仅跳过赋值，该位置可能保留上一轮的数据，使不应参与计算的项继续产生贡献。

零填充的三个动作

加载 A ：使用 A 自身的边界条件，越界写零。

加载 B ：使用 B 自身的边界条件，越界写零。

写回 C ：仅当输出坐标有效时写入全局内存。

两个同步点仍由整个线程块共同执行。

4.2.2 例一：输出有效，但最后一轮输入不完整

取 $M = K = N = 3$ 、 $T = 2$ 。线程块 $(b_x, b_y) = (0, 0)$ 负责左上角四个有效输出，共执行 $Q = 2$ 轮。第二轮 $q = 1$ 对应全局内积下标 $j = 2, 3$ ，其中 $j = 3$ 超出有效输入范围。共享数组应为

$$A_s = \begin{bmatrix} A_{0,2} & 0 \\ A_{1,2} & 0 \end{bmatrix}, \quad B_s = \begin{bmatrix} B_{2,0} & B_{2,1} \\ 0 & 0 \end{bmatrix}. \quad (12)$$

在 $k = 0$ 时，四个线程分别增加 $A_{r,2}B_{2,c}$ ；在 $k = 1$ 时，共享行的第二个元素与共享列的第二个元素均为零，贡献为零。以 $C_{0,1}$ 为例，全部两轮累加为

$$\underbrace{A_{0,0}B_{0,1} + A_{0,1}B_{1,1}}_{q=0} + \underbrace{A_{0,2}B_{2,1} + 0 \cdot 0}_{q=1}.$$

这个线程的输出位置始终有效，但它仍需要对最后一轮输入进行边界处理。

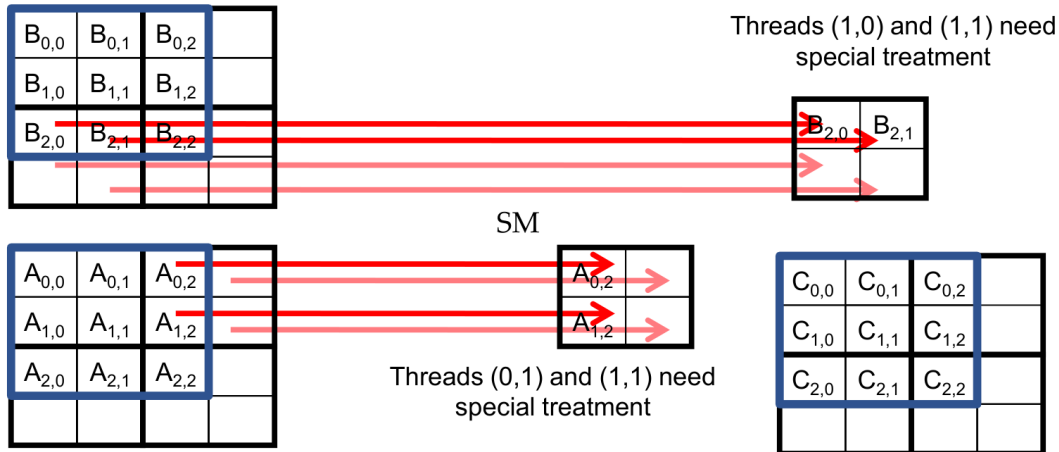


图 6 3×3 矩阵使用 2×2 分块时，第二轮输入块包含缺失位置。图中的线程编号采用（行，列）次序。

4.2.3 例二：输出越界的线程仍承担加载任务

仍取 3×3 矩阵与 2×2 分块。线程块 $(b_x, b_y) = (1, 1)$ 覆盖输出坐标 $r, c \in \{2, 3\}$ ，只有 $C_{2,2}$ 有效。在第一轮 $q = 0$ ，四个线程的任务如下；线程坐标均按 (t_x, t_y) 书写。

线程	输出位置	加载 A	加载 B
(0,0)	$C_{2,2}$ 有效	$A_{2,0}$	$B_{0,2}$
(1,0)	$C_{2,3}$ 越界	$A_{2,1}$	越界，写零
(0,1)	$C_{3,2}$ 越界	越界，写零	$B_{1,2}$
(1,1)	$C_{3,3}$ 越界	越界，写零	越界，写零

于是共享数组为

$$A_s = \begin{bmatrix} A_{2,0} & A_{2,1} \\ 0 & 0 \end{bmatrix}, \quad B_s = \begin{bmatrix} B_{0,2} & 0 \\ B_{1,2} & 0 \end{bmatrix}.$$

唯一负责有效输出的线程 (0,0)，需要线程 (1,0) 加载的 $A_{2,1}$ 和线程 (0,1) 加载的 $B_{1,2}$ ，才能计算

$$A_{2,0}B_{0,2} + A_{2,1}B_{1,2}.$$

第二轮再增加 $A_{2,2}B_{2,2}$ ，最终得到完整的 $C_{2,2}$ 。因此，应当保留输出越界线程的协作加载与同步，只在写回时排除其输出。

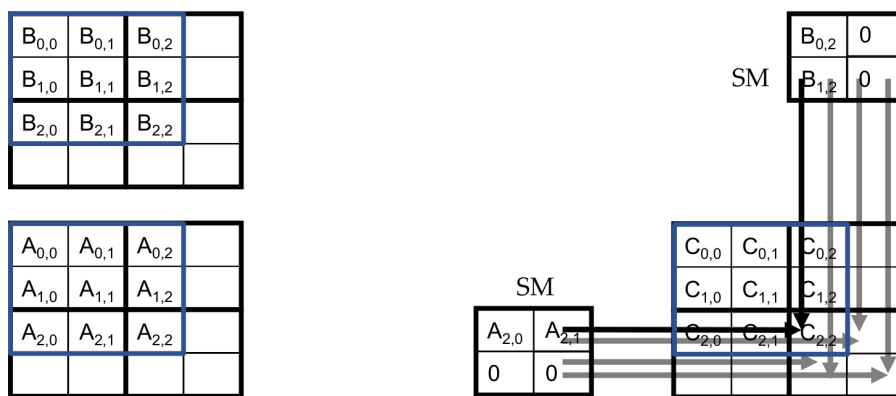


图 7 右下角输出块只含一个有效输出，但该输出仍需要其他线程协作加载输入。

4.3 支持一般尺寸的完整实现

4.3.1 核函数与启动配置

代码 2 把三类边界判断、零填充和向上取整组合起来。假设 M, K, N 为正整数，矩阵使用行主序，输入和输出数组已经按式 (8) 分配。

代码 2 一般尺寸的分块矩阵乘法；加载与同步由全部线程参与。

```

1  #include <cuda_runtime.h>
2  #define TILE_DIM 16
3
4  __global__ void matmul_tiled(
5      const float* A, const float* B, float* C,
6      int M, int K, int N)
7  {
8      __shared__ float A_s[TILE_DIM][TILE_DIM];
9      __shared__ float B_s[TILE_DIM][TILE_DIM];
10     const int tx = threadIdx.x;
11     const int ty = threadIdx.y;
12     const int row = blockIdx.y * TILE_DIM + ty;
13     const int col = blockIdx.x * TILE_DIM + tx;
14     const int numTiles = (K - 1) / TILE_DIM + 1;
15     float sum = 0.0f;
16
17     for (int q = 0; q < numTiles; ++q) {
18         const int aCol = q * TILE_DIM + tx;
19         const int bRow = q * TILE_DIM + ty;
20         A_s[ty][tx] = (row < M && aCol < K)
21             ? A[row * K + aCol] : 0.0f;
22         B_s[ty][tx] = (bRow < K && col < N)
23             ? B[bRow * N + col] : 0.0f;
24         __syncthreads();
25
26         for (int k = 0; k < TILE_DIM; ++k) {
27             sum += A_s[ty][k] * B_s[k][tx];
28         }
29         __syncthreads();
30     }
31     if (row < M && col < N) {
32         C[row * N + col] = sum;
33     }
34 }

```

启动时，网格横向覆盖 N 列，纵向覆盖 M 行：

```

dim3 block(TILE_DIM, TILE_DIM);
dim3 grid((N - 1) / TILE_DIM + 1,
          (M - 1) / TILE_DIM + 1);
matmul_tiled<<<grid, block>>>(d_A, d_B, d_C, M, K, N);

```

参数顺序为 M, K, N ，与 A 的行数、内积长度、 B 的列数一致。

4.3.2 为什么零填充保持结果正确

对输入在原矩阵范围外定义零扩展 $\tilde{A}$ 、 $\tilde{B}$ 。当 $0 \leq r < M$ 、 $0 \leq c < N$ 时，代码实际计算

$$\sum_{q=0}^{Q-1} \sum_{k=0}^{T-1} \tilde{A}_{r,qT+k} \tilde{B}_{qT+k,c} = \sum_{j=0}^{QT-1} \tilde{A}_{r,j} \tilde{B}_{j,c} \quad (13)$$

$$= \sum_{j=0}^{K-1} A_{r,j} B_{j,c} = C_{r,c}. \quad (14)$$

由于 $QT \geq K$ ，有效内积项全部被覆盖；多出来的项恰好为零。此推导解释了为何内积循环仍可固定执行 T 步。零扩展只是在共享加载阶段实现的逻辑含义，无需为完整矩阵额外分配补零后的全局数组。

4.3.3 可选的计算判断与同步位置

对于无有效输出的线程，零填充使其相应共享行或共享列为零，部分和不影响任何有效输出。可以继续执行内积循环，也可以仅让有效输出线程执行乘加，以减少这些线程的无用运算。该调整的实际性能收益取决于分支和省去运算的开销。

若增加计算条件，正确的局部结构如下：

```
__syncthreads();
if (row < M && col < N) {
    for (int k = 0; k < TILE_DIM; ++k) {
        sum += A_s[ty][k] * B_s[k][tx];
    }
}
__syncthreads();
```

两个同步点都在按线程变化的条件之外。将输出越界线程提前移出分块循环，会丢失它们仍可能承担的输入加载任务；将同步放入仅部分线程执行的分支，也不满足块内同步要求。

4.3.4 边界分支的工作量范围

边界条件可能造成分支分歧 (*branch divergence*)。输出的末行、末列分块会包含输出越界线程；当 K 不能被 T 整除时，每个输出块的最后一轮也会处理内积方向的缺失输入。这一分类由三类边界谓词直接得到。

在矩阵各维度相对 T 都很大时，边缘区域和最后一轮输入只占较小比例；小矩阵或高度

不规则的尺寸则可能有明显边界开销。该判断延续大矩阵边界影响较小的定性分析，具体占比仍由 M, N, K, T 决定。

5 分块收益与资源约束

5.1 用数据流量解释计算收益

5.1.1 一个分块轮次的加载量与计算量

以下推导展开分块减少全局访问的数量关系。采用 $N \times N$ 方阵，且 T 整除 N ；按代码中的全局输入加载计数，暂时忽略缓存效果、输出流量、同步和其他执行开销。

一个线程块的一轮计算，需要加载两个 $T \times T$ 输入块，共 $2T^2$ 个 `float`，即 $8T^2$ 字节。该轮计算 T^2 个输出部分和，每个包含 T 次乘加；按每次乘加两个浮点运算计，计算量为 $2T^3$ FLOP。于是，计算强度 (*arithmetic intensity*) 为

$$I_T = \frac{2T^3 \text{ FLOP}}{8T^2 \text{ B}} = \frac{T}{4} \text{ FLOP/B.} \quad (15)$$

等价地，每个浮点运算需要 $4/T$ 字节的全局输入数据。 T 增大时，同一输入被更多输出复用。

对比不使用共享复用的情况： T^2 个线程各自为 T 个内积步加载两次输入，共有 $2T^3$ 次输入元素加载；协作加载只需 $2T^2$ 次。因此，当前线程块这一轮的输入加载量减少为原来的 $1/T$ 。

5.1.2 整个矩阵乘法的流量

不使用共享复用时， N^2 个输出各需 $2N$ 次输入加载，总数为 $2N^3$ 。分块后，有 $(N/T)^2$ 个输出块，每块执行 N/T 轮，每轮加载 $2T^2$ 个输入元素，得到

$$L_T = \left(\frac{N}{T}\right)^2 \frac{N}{T} 2T^2 = \frac{2N^3}{T}. \quad (16)$$

当前方法的复用发生在线程块内部；不同输出块仍分别加载它们需要的输入块。上述计数保留了这一点。

将输出写回加入流量模型

每个有效输出最终写回一次，故输出流量为 $4N^2$ 字节。把输入与输出合并，可得到更完整的计算强度近似：

$$I_T^{\text{in+out}} = \frac{2N^3}{8N^3/T + 4N^2} = \frac{NT}{4N + 2T} \text{ FLOP/B.}$$

当 N 相对 T 很大时，该式趋近 $T/4$ 。这一补充用于说明忽略输出写回的近似条件，其余缓存和执行开销仍未包含。

5.2 带宽瓶颈如何转向计算瓶颈

令 P_{peak} 表示计算峰值， B_{mem} 表示全局内存带宽。用计算峰值和带宽支持能力作为两项上界，可写成

$$P_T \leq \min(P_{\text{peak}}, B_{\text{mem}} I_T). \quad (17)$$

这是一种用于解释瓶颈转移的简化模型。代入示例的 $P_{\text{peak}} = 1000 \text{ GFLOP/s}$ 、 $B_{\text{mem}} = 150 \text{ GB/s}$ ，有

$$B_{\text{mem}} I_T = 37.5T \text{ GFLOP/s.}$$

T	输入字节/FLOP	带宽支持能力	同时考虑计算峰值
1	4	37.5 GFLOP/s	37.5 GFLOP/s
16	0.25	600 GFLOP/s	600 GFLOP/s
32	0.125	1200 GFLOP/s	1000 GFLOP/s

16×16 分块把带宽支持能力提高到 600 GFLOP/s 。 32×32 分块的输入供给可以支持 1200 GFLOP/s ，已经超过示例 GPU 的计算峰值，所以该简化模型的限制转为计算能力。

这些数值描述带宽可以支撑的运算量及理论上界。实际执行还受到输出流量、缓存效果、同步、占用率和寄存器压力影响；这些因素正是从流量模型走向实际性能时需要补充的项。

5.3 分块尺寸与驻留资源

5.3.1 每个线程块需要多少资源

每线程一个输出、两个共享输入块的基本实现中，线程数和共享内存用量为

$$n_{\text{threads}} = T^2, \quad S_{\text{block}} = 2T^2 \text{sizeof(float)} = 8T^2 \text{ B.} \quad (18)$$

较大的分块带来更多复用，也同时增大线程块和共享内存占用。资源分配会限制一个流多处理器(*streaming multiprocessor, SM*)上可同时驻留的线程块数量，进而影响占用率(*occupancy*)。

占用率描述驻留 warp 相对于硬件可容纳 warp 数的比例。对这里的 16×16 、 32×32 规则线程块，可通过驻留线程数与线程上限比较其资源占用水平。资源容量给出驻留数量的上界，实际驻留状态还取决于核函数的资源需求。

5.3.2 示例：共享容量限制与线程容量限制

采用第一代 Maxwell 的示例条件：每个 SM 有 64 kB 共享内存，每线程块最多使用 48 kB，每个 SM 最多驻留 2048 个线程。以下存储容量换算使用 $1 \text{ kB} = 1024 \text{ B}$ 。

资源或计数	$T = 16$	$T = 32$
每块线程数 T^2	256	1024
每块共享内存 $8T^2$	2048 B = 2 kB	8192 B = 8 kB
仅按共享容量限制的块数	$64/2 = 32$	$64/8 = 8$
仅按线程容量限制的块数	$2048/256 = 8$	$2048/1024 = 2$
上述两项合并的块数上限	8	2
相应驻留线程数上限	$8 \times 256 = 2048$	$2 \times 1024 = 2048$
每线程两次加载的计数	$8 \times 256 \times 2 = 4096$	$2 \times 1024 \times 2 = 4096$

两个分块尺寸在这一估算中都暴露出 4096 个待加载操作。 32×32 分块虽然允许驻留的线程块更少，但每块线程更多，合计线程数相同；其输入复用倍数则更高。这里的加载数是按驻留线程和每线程两次加载得到的并行性估算。

5.3.3 寄存器与其他上限共同决定驻留数量

共享内存和线程数是两项直接可见的限制，寄存器数量及每 SM 的线程块数量上限也会约束驻留。令 S_{SM} 为每 SM 共享内存容量， R_{SM} 为寄存器数量， R_{thread} 为每线程寄存器用量， n_{max} 与 B_{max} 分别为每 SM 的线程数和块数上限。可用下面的理想化上界概括最紧资源约束：

$$B_{\text{active}} \leq \min \left\{ B_{\text{max}}, \left\lfloor \frac{n_{\text{max}}}{T^2} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{8T^2} \right\rfloor, \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}} T^2} \right\rfloor \right\}. \quad (19)$$

该式将资源容量除以每块需求，表达“由最先耗尽的资源限制驻留”的原则；实际资源分配粒度未包含在内。每块资源需求还必须满足设备对单个线程块的限制。

表 1 GPU 资源规格示例；M40 属于第二代 Maxwell。

GPU 型号	K40	M40	P100	V100	A100	H100
微架构	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
计算能力	3.5	5.2	6.0	7.0	8.0	9.0
SM 数量	15	24	56	80	108	132
每 SM 核心数	192	128	64	64	64	128
每 SM 最大线程数	2048	2048	2048	2048	2048	2048
每 SM 最大块数	16	32	32	32	32	32
每块最大线程数	1024	1024	1024	1024	1024	1024
每 SM 寄存器数	64K	64K	64K	64K	64K	64K
每 SM 共享内存	48KB	96KB	64KB	96KB	164KB	228KB

5.3.4 如何理解分块尺寸的取舍

分块边长 T 同时影响两组数量：复用次数随 T 增长，而每块线程数与共享内存需求随 T^2 增长。选择尺寸时，需要把输入复用、可驻留线程块数量、寄存器消耗和同步代价放在同一个执行模型中考虑。

基本实现中， 16×16 对应 256 个线程， 32×32 对应 1024 个线程；后者已经达到表 1 中列出的每块最大线程数。提高复用的空间因此也受到线程映射方式约束。具体核函数在实际设备上的较优尺寸仍需由测量确定。

6 共享内存接口与分块思想的推广

6.1 动态共享内存

6.1.1 将容量配置推迟到核函数启动时

静态共享数组的大小写在声明中，例如 `A_s[TILE_DIM][TILE_DIM]`。当所需容量需要在启动时确定，可以使用**动态共享内存** (*dynamic shared memory*):

```
extern __shared__ float A_s[];
```

核函数启动配置的第三个参数 `smemPerBlock` 指定每个线程块申请的动态共享内存字节数。配置形式为 `<<<nBlocks, nThreadsPerBlock, smemPerBlock>>>`。该容量按线程块分别提供，单位是字节。

6.1.2 将一段共享区域划分为两个输入块

以下局部片段用于把动态共享内存接口连接到两个输入块的存储需求。设运行时的分块边长为 T ，可以在核函数内部声明一段共享区域，再把它分成前后两段：

```
extern __shared__ float shared[];  
float* A_s = shared;  
float* B_s = shared + T * T;
```

两个输入块各占 T^2 个 `float`。采用一维共享指针后，局部位置 (t_y, t_x) 使用偏移 $t_y T + t_x$ ；计算时的两个偏移分别为 $t_y T + k$ 和 $kT + t_x$ 。对应的主机端容量计算为：

```
size_t smemPerBlock = 2 * T * T * sizeof(float);
```

分区方式只改变共享数组的表示和容量配置；协作加载、两次同步、零填充和输出写回的职责保持相同。该局部片段为存储布局说明，完整矩阵乘法实现采用代码 2 的静态共享数组版本。

6.2 Hopper 的相关功能

6.2.1 分布式共享内存

分布式共享内存(*distributed shared memory*)允许同一线程块集群(*thread block cluster*)内的线程块访问彼此的共享内存。它从 Hopper 架构开始提供,将访问范围扩展到集群中的线程块。前面的基本分块核函数使用各线程块自己的共享数组。

6.2.2 张量内存加速器

张量内存加速器(*Tensor Memory Accelerator, TMA*)支持全局内存与共享内存之间的数据异步传输。它可用于分块算法的输入搬运阶段。

6.3 CPU 缓存分块

6.3.1 相同的局部性,不同的存储管理方式

CPU 上也可以通过分块重排计算顺序。缓存分块(*cache blocking*)使一小组数据在较短的执行区间内被反复使用,从而建立时间局部性(*temporal locality*),提高数据仍在缓存中时被再次访问的机会。

GPU 基本实现由程序把输入块显式放入共享内存,并通过线程块同步协调使用;CPU 示例依靠缓存复用,通过循环顺序集中对当前分块的访问。二者都将计算组织为“小块数据,多次使用”。

6.3.2 三层分块循环与三层元素循环

CPU 示例对输出行、输出列和内积方向分别划分分块。外层三重循环选择输出块与当前输入块，内层三重循环计算该输入块对输出块的贡献。代码 3 是该循环结构的函数实现；假设 $N > 0$ 、 $T > 0$ ，且 T 整除 N 。

代码 3 CPU 缓存分块矩阵乘法，保持每个内积块的部分和累加顺序。

```

1 void matmul_cpu_tiled(
2     const float* A, const float* B, float* C, int N, int T)
3 {
4     for (int rt = 0; rt < N / T; ++rt) {
5         for (int ct = 0; ct < N / T; ++ct) {
6             for (int qt = 0; qt < N / T; ++qt) {
7                 for (int row = rt * T;
8                     row < (rt + 1) * T; ++row) {
9                     for (int col = ct * T;
10                        col < (ct + 1) * T; ++col) {
11                         float sum = 0.0f;
12                         for (int j = qt * T;
13                             j < (qt + 1) * T; ++j) {
14                             sum += A[row * N + j]
15                                 * B[j * N + col];
16                         }
17                         if (qt == 0) {
18                             C[row * N + col] = sum;
19                         } else {
20                             C[row * N + col] += sum;
21                         }
22                     }
23                 }
24             }
25         }
26     }
27 }

```

6.3.3 初始化与部分和的归属

对于固定的 $C_{r,c}$ ，每个内积块产生一个新的局部变量 **sum**。第一块通过赋值初始化 $C_{r,c}$ ，后续块通过 **+=** 累加。因此，初始的 C 数组内容无需参与计算。

GPU 基本核函数把一个输出的 **sum** 保持在线程寄存器中，跨越全部输入分块，最后统一写回；这里的 CPU 循环先计算当前内积块对整个输出块的贡献，跨内积块的部分和保存在 C 中。两种写法都实现式 (5)，但部分和保留的位置与循环组织不同。

7 关键结论与公式汇总

7.1 机制之间的联系

7.1.1 复用决定分块的收益

矩阵乘法的同一个输入会服务于多个输出。一个 $T \times T$ 线程块先协作加载 $2T^2$ 个输入，再计算 $2T^3$ 个浮点运算，将块内输入复用转化为约 T 倍的全局加载量缩减。

7.1.2 同步决定共享缓冲区的正确使用

共享数组在空间上由多个线程共同使用，在时间上由多个分块轮次复用。第一次屏障保护当前数据的完整加载，第二次屏障保护当前数据的完整使用。两次屏障分别对应这两类依赖。

7.1.3 边界处理决定协作任务能否保留

每个线程的 A 加载、 B 加载和 C 写回具有不同的有效性条件。加载阶段写零，使内积循环保持统一；输出阶段做有效性判断，使越界线程保留协作职责而不写入非法输出位置。

7.1.4 资源约束决定复用能否转化为性能

分块增大使计算强度线性增长，同时使每块线程数和共享内存需求按平方增长。带宽、计算峰值、驻留线程块数量以及寄存器需求，共同影响最终的执行速度。

7.2 关键公式

以下使用 $A \in \mathbb{R}^{M \times K}$ 、 $B \in \mathbb{R}^{K \times N}$ 、 $C \in \mathbb{R}^{M \times N}$ ，分块边长为 T 。

对象	表达式
线程对应输出	$r = b_y T + t_y, \quad c = b_x T + t_x。$
输出网格尺寸	$n_x = \lceil N/T \rceil, \quad n_y = \lceil M/T \rceil。$
内积方向轮数	$Q = \lceil K/T \rceil。$
A 加载地址与条件	$rK + qT + t_x; \quad r < M \text{ 且 } qT + t_x < K。$
B 加载地址与条件	$(qT + t_y)N + c; \quad qT + t_y < K \text{ 且 } c < N。$
共享内积	$\sum_{k=0}^{T-1} A_s[t_y][k] B_s[k][t_x]。$
输出地址与条件	$rN + c; \quad r < M \text{ 且 } c < N。$
每块线程与共享容量	T^2 个线程, $8T^2$ 字节共享内存。
理想输入计算强度	$I_T = T/4 \text{ FLOP/B}。$
简化性能上界	$P_T \leq \min(P_{\text{peak}}, B_{\text{mem}} I_T)。$

计算强度的单精度假设、浮点计数与流量模型见第 5 节。